Final Project

Jeopardy Value Prediction

Rohit Viswanathan, Jacob Fingerman, Aryan Shukla, Syed Khaled

Background

```
library(dplyr)
```

```
Warning: package 'dplyr' was built under R version 4.2.3
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
jeopardy <- read.csv('JEOPARDY_CSV.csv')
```

For our project, we are working with archival show data from the classic game show Jeopardy. On Jeopardy, three contestants compete to answer a series of trivia questions, earning money for every question that they answer correctly. The first contestant to buzz in and answer a question correctly gets to choose the next question from the gameboard. If a contestant buzzes in but their answer is incorrect, the question's value is deducted from their total earnings. A standard episode of Jeopardy consists of three rounds: Jeopardy, Double Jeopardy, and Final Jeopardy. For each of the first two rounds, the Jeopardy board is divided into six unique categories with five questions each of increasing value. This is an example of a typical gameboard from the Jeopardy round:

In the Double Jeopardy round, the structure of the gameboard remains the same, except that the value of all the questions is doubled. At the end of the Double Jeopardy round, all contestants with a positive amount of money are allowed to compete in Final Jeopardy. Final Jeopardy consists of a single question in a new category. After the category is revealed to the contests, they may wager any amount of their money to stake on the final question. Once all contestants place their wagers, the final jeopardy question is revealed and the contestants write down their answers. The wagered amount for each contestant is added to their earnings if they answer correctly, but deducted if they answer incorrectly. The contestant with the most money at the end of the game is deemed the winner and walks home with their earned amount.

- Show Number: The number of the aired episode

- Air Date: Date the episode aired
- Round: The round in which the question was asked (Jeopardy!, Double Jeopardy!, Final Jeopardy!)
- Category: The question's category
- Value: How much money earned from answering the question
- Question: The Question that is being asked
- Answer: The Answer to the question

Goal of the Project

The goal of our project is to utilize natural language processing techniques to see if there is a correlation between the text used in the Question and Answer for a given Jeopardy Question and the Question's value on the gameboard.

Data Cleaning

Before we could get to work on modeling, there were a few issues we had to address about our dataset. Our first order of business was to get rid of all the Final Jeopardy questions. Then we turned our Jeopardy values from strings to numeric by removing the $ sign and the commas. and This is because, unlike the questions in the Jeopardy and Double Jeopardy rounds, Final Jeopardy questions have no standard associated value; they are all random and based on contestant wagers. Since there is no consistency from one final jeopardy to another, our text analysis will not work for predicting the value of these questions. Next, we had to remove hyperlinks from all of the questions that contained a visual or audio cue. Rather than plain text, these were jeopardy questions that relied on a video clip or an audio snippet when the question appeared on air.Therefore, in our database, all of these questions contained a hyperlink to the jeopardy write where you could listen to or watch the clip. Luckily, these questions had a description that was accompanied by the hyperlink, so the text data was able to be retained while disposing of the links. After that, we had to scale up the value for all Jeopardy shows that occurred before 11/26/2001. Prior to that date, questions in the Jeopardy round were worth 100/200/300/400/500, and questions in the Double Jeopardy round were worth 200/400/600/800/1000, exactly half of what they are worth on current day Jeopardy. So, we scaled up the data from the old shows so that all of our data would follow the same scale.

Our toughest challenge in the cleanup process was dealing with the daily doubles. The Daily Double is a jeopardy question that looks like an ordinary question from the outside, but upon being selected by a contestant, it is revealed to be the Daily Double. In the case of the Daily Double, the associated value of the question is discarded and the contestant who chose the question is able to wager an amount they would like to answer for. The Daily Double occurs once in the jeopardy round and twice in the double jeopardy round. Unfortunately for us, the dataset does not have a method for marking which questions are daily doubles or not. Thus, it was very difficult for us to go through and remove daily doubles because they happened three times a show and the only way to recognize them was by seeing that a question's value deviated from the standard value scale of 200/400/600/800/1000 or 400/800/1200/1600/2000 for double jeopardy. Our method for dealing with these wagered amounts was to filter out all values that had less than

1,000 occurrences. That way, we would be left with only the values that follow the value scale. This does leave room for some error, as it is possible that a contestant chose a question with a value of 600 on the game board but then decided to wager 400 on it, meaning it follows the value scale but the original price is lost, muddying our data. Alas, this was the best fix we were able to implement for the issue.

```
jeopardy_data <- jeopardy
# removing dollar signs and commas from the numbers and changing them to numeric
jeopardy_data$Value <- as.numeric(gsub("[\\$,]", "", jeopardy_data$Value))
```

```
Warning: NAs introduced by coercion
```

```
# Turning the NAs (previously "None") into 0s
jeopardy_data$Value[is.na(jeopardy_data$Value)] <- 0
```

```
#Removing Video/Audio Queues

library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.2.3

Warning: package 'ggplot2' was built under R version 4.2.3

Warning: package 'tibble' was built under R version 4.2.3

Warning: package 'tidyr' was built under R version 4.2.3

Warning: package 'readr' was built under R version 4.2.3

Warning: package 'purrr' was built under R version 4.2.3

Warning: package 'stringr' was built under R version 4.2.2

Warning: package 'forcats' was built under R version 4.2.3

Warning: package 'lubridate' was built under R version 4.2.3

── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ forcats   1.0.0     ✓ readr     2.1.4
✓ ggplot2   3.5.1     ✓ stringr   1.5.0
✓ lubridate 1.9.3     ✓ tibble    3.2.1
✓ purrr     1.0.2     ✓ tidyr     1.3.0
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
errors
```

```
jeopardy_data <- jeopardy_data %>%
  mutate(
```

```
    Question = str_replace_all(Question, "<.*?>", "")
  ) %>%
  mutate(
    Question = str_replace_all(Question, "<a.*?>", "")
  ) %>%
  mutate(
    Question = str_replace_all(Question, "</a.*?>", "")
  )
```

```
jeopardy_adjusted_data <- jeopardy_data %>%
  mutate(Value = as.numeric(gsub("[\\$,]", "", Value))) %>%
  drop_na(Value) %>%
  filter(Value != 0)
```

```
library(tidytext)
```

```
Warning: package 'tidytext' was built under R version 4.2.3
```

```
data("stop_words")

word_counts <- jeopardy_adjusted_data %>%
  unnest_tokens(word, Question) %>%           # Tokenize the 'Question' column
  anti_join(stop_words, by = "word") %>%      # Remove stop words
  count(word, sort = TRUE)
```

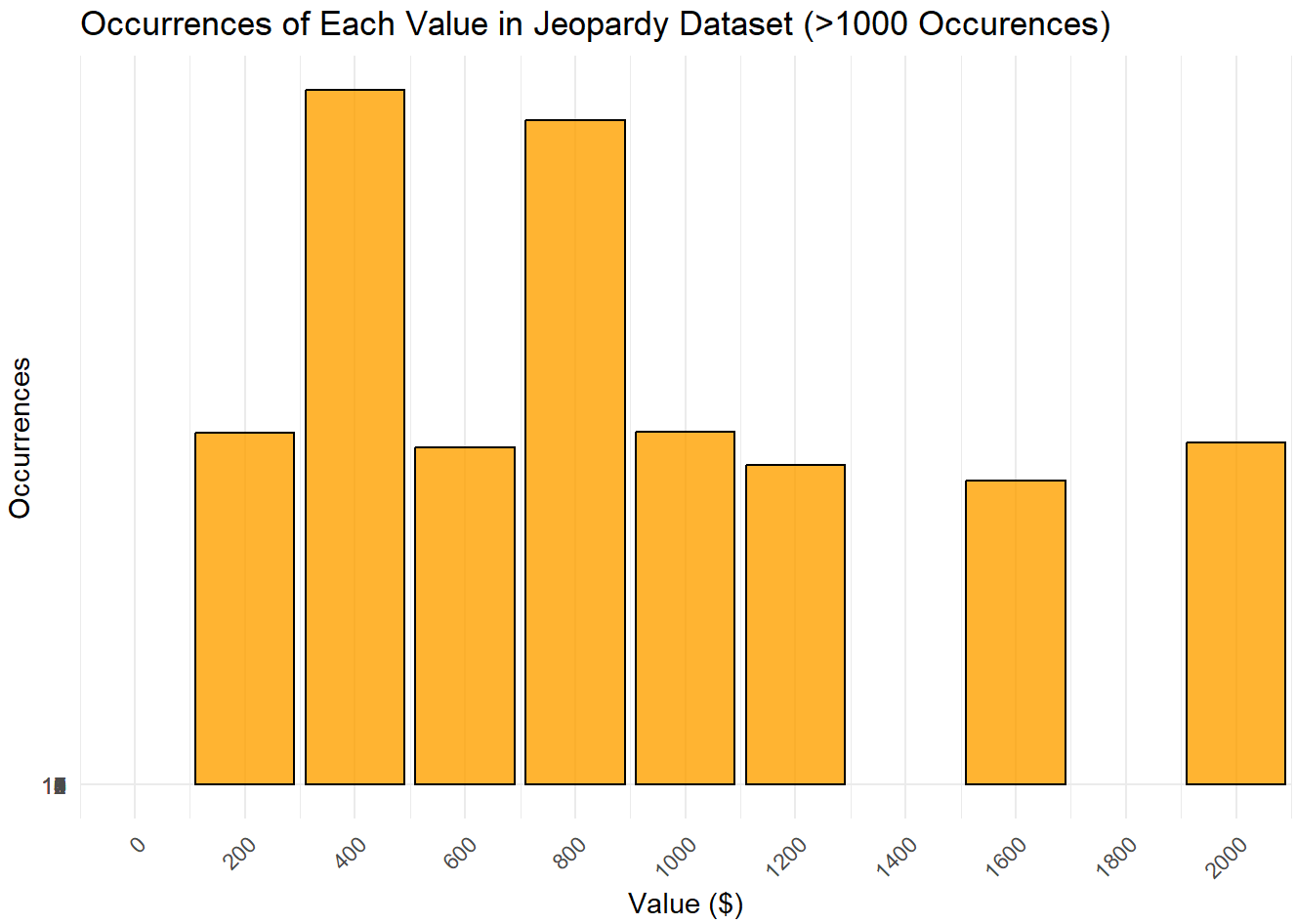
```
#Filtered out values with less than 1000 occurences because those are Daily Double values.

value_counts <- jeopardy_adjusted_data %>%
  group_by(Value) %>%
  summarize(Occurrences = n()) %>%
  ungroup() %>%
  filter(Occurrences >= 1000)
```

Exploratory Data Analysis

Now that all the data had been cleaned and was ready for use, we were able to do some exploratory data analysis. The first thing we wanted to see was the distribution of the questions across the different values after handling the Daily Double.

```
jeopardy_adjusted_data <- jeopardy_adjusted_data %>%
  mutate(Value = ifelse(Air.Date < as.Date("2001-11-26"), Value * 2, Value))
#Rerunning Rohit's Value Count Plot with the adjusted data

value_counts_adjusted <- jeopardy_adjusted_data %>%
  group_by(Value) %>%
  summarize(Occurrences = n()) %>%
```

```
  ungroup() %>%
  filter(Occurrences >= 1000)

ggplot(value_counts_adjusted, aes(x = Value, y = Occurrences)) +
  geom_col(fill = "orange", color = "black", alpha = 0.8) +
  labs(
    title = "Occurrences of Each Value in Jeopardy Dataset (>1000 Occurences)",
    x = "Value ($)",
    y = "Occurrences"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = seq(0, 2000, by = 200)) +
  scale_y_continuous(breaks = seq(0, 12, by = 1)) +
  coord_cartesian(xlim = c(0, 2000)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

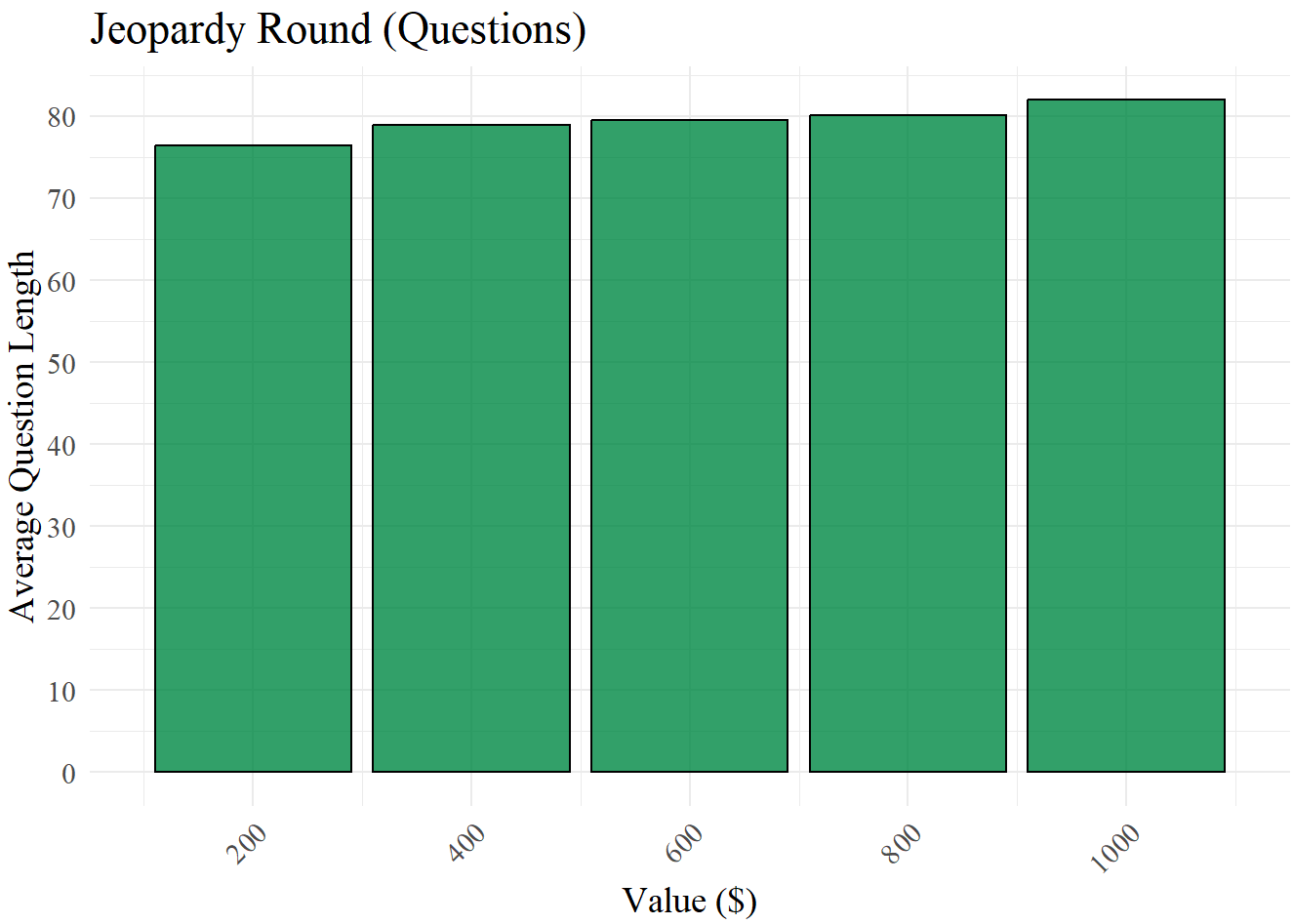


The resulting graph showed that the handling of the daily double went fairly well, as the distribution of questions across the different values was roughly uniform with the exception of 400 and 800. This was to be expected as Jeopardy follows a 200/400/600/800/1000 value scale and double jeopardy follows a 400/800/1200/1600/2000 value scale, so 400 and 800 should have about double the number of occurrences since they appear in both rounds.

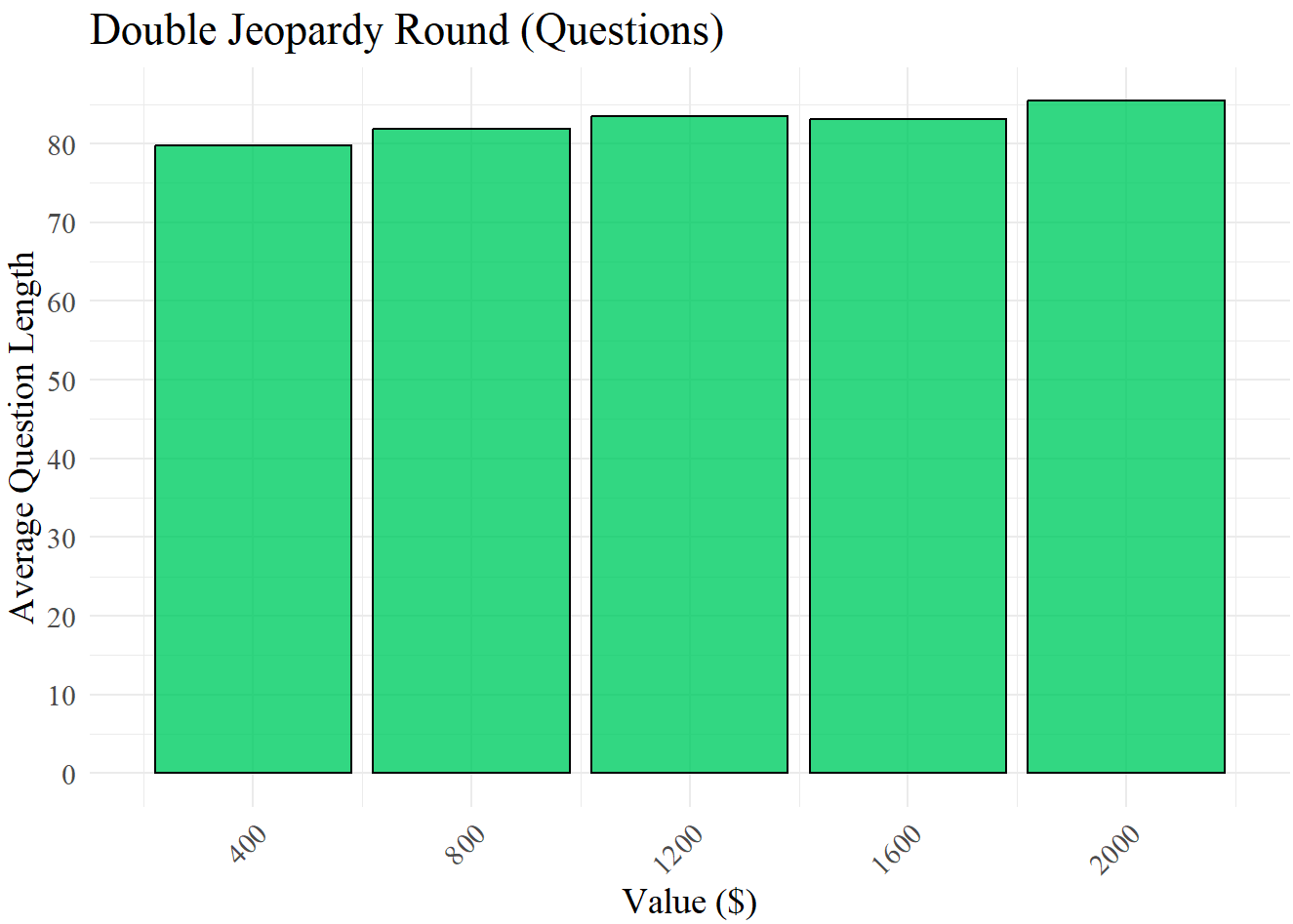
The next thing we wanted to explore was if there was any correlation between the length of a question and its value.

```
library(tidyverse)


# Calculate the average length of questions for each round and value
average_question_length <- jeopardy_adjusted_data %>%
  mutate(
    QuestionLength = nchar(as.character(Question)),
    Value = as.numeric(gsub("[\\$,]", "", Value))
  ) %>%
  drop_na(Value) %>%
  filter(Value != 0) %>%
  group_by(Round, Value) %>%
  summarize(
    AverageQuestionLength = mean(QuestionLength, na.rm = TRUE),
    Occurrences = n(),
    .groups = "drop"
  ) %>%
  filter(Occurrences >= 1000)
# Separate datasets for Jeopardy and Double Jeopardy rounds
jeopardy_avg_q <- average_question_length %>%
  filter(Round == "Jeopardy!")

double_jeopardy_avg_q <- average_question_length %>%
  filter(Round == "Double Jeopardy!")

# Plot for Jeopardy round
plot_jeopardy_q <- ggplot(jeopardy_avg_q, aes(x = Value, y = AverageQuestionLength)) + geom_col(f
  labs(
    title = "Jeopardy Round (Questions)",
    x = "Value ($)",
    y = "Average Question Length"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = seq(0, 1000, by = 200)) + # Set x-axis breaks up to 1000
  scale_y_continuous(breaks = seq(0, 100, by = 10)) +
  coord_cartesian(xlim = c(100, 1100)) +                # Cap x-axis at 1000
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
  theme(text=element_text(size=14,  family="serif"))

# Plot for Double Jeopardy round
plot_double_jeopardy_q <- ggplot(double_jeopardy_avg_q, aes(x = Value, y = AverageQuestionLength)
  geom_col(fill = "springgreen3", color = "black", alpha = 0.8) +
  labs(
    title = "Double Jeopardy Round (Questions)",
    x = "Value ($)",
    y = "Average Question Length"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = seq(0, 2000, by = 400)) +
  scale_y_continuous(breaks = seq(0, 100, by = 10)) +
```

```
  coord_cartesian(xlim = c(200, 2200)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
  theme(text=element_text(size=14,  family="serif"))

plot_jeopardy_q
```



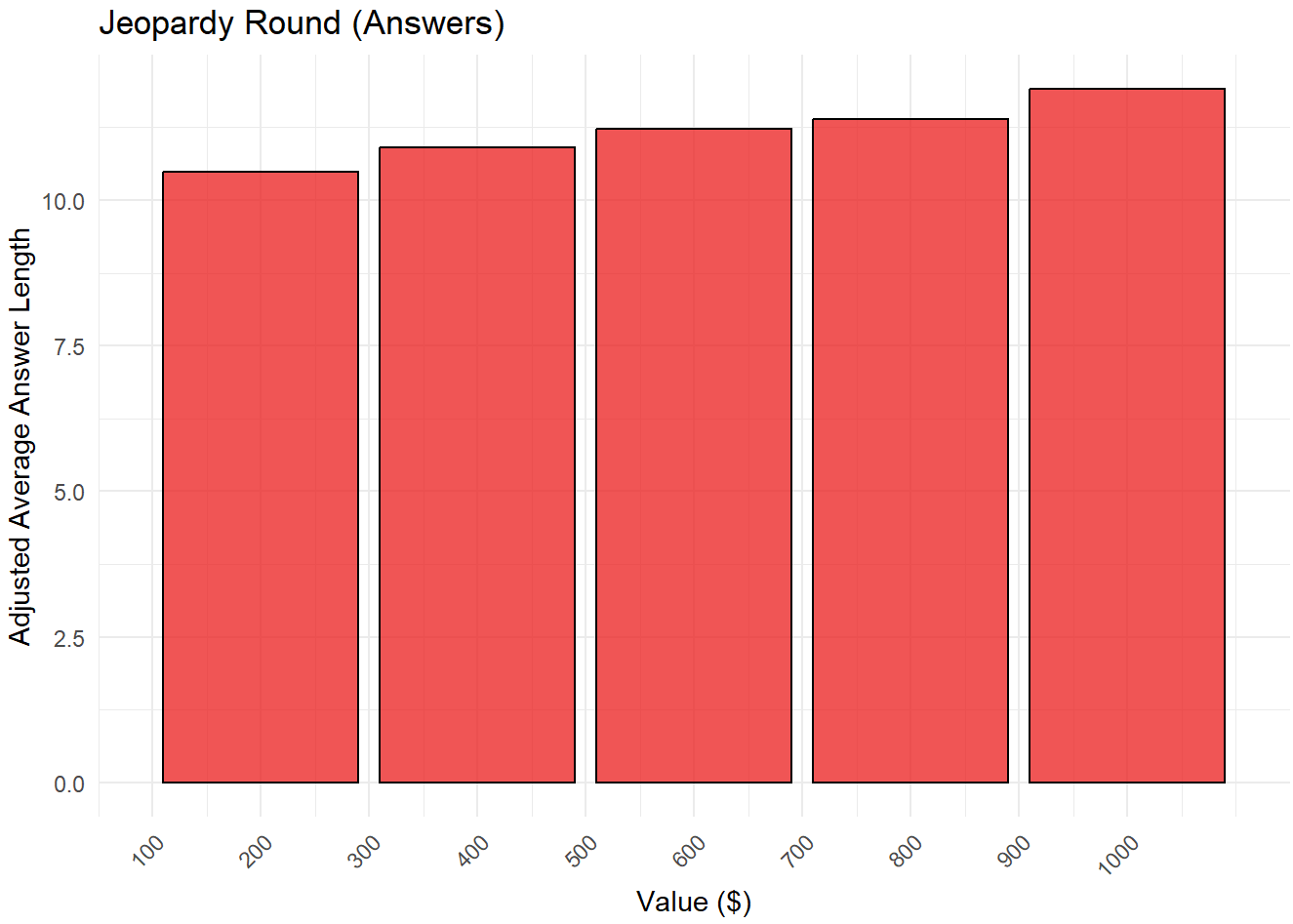

```
plot_double_jeopardy_q
```



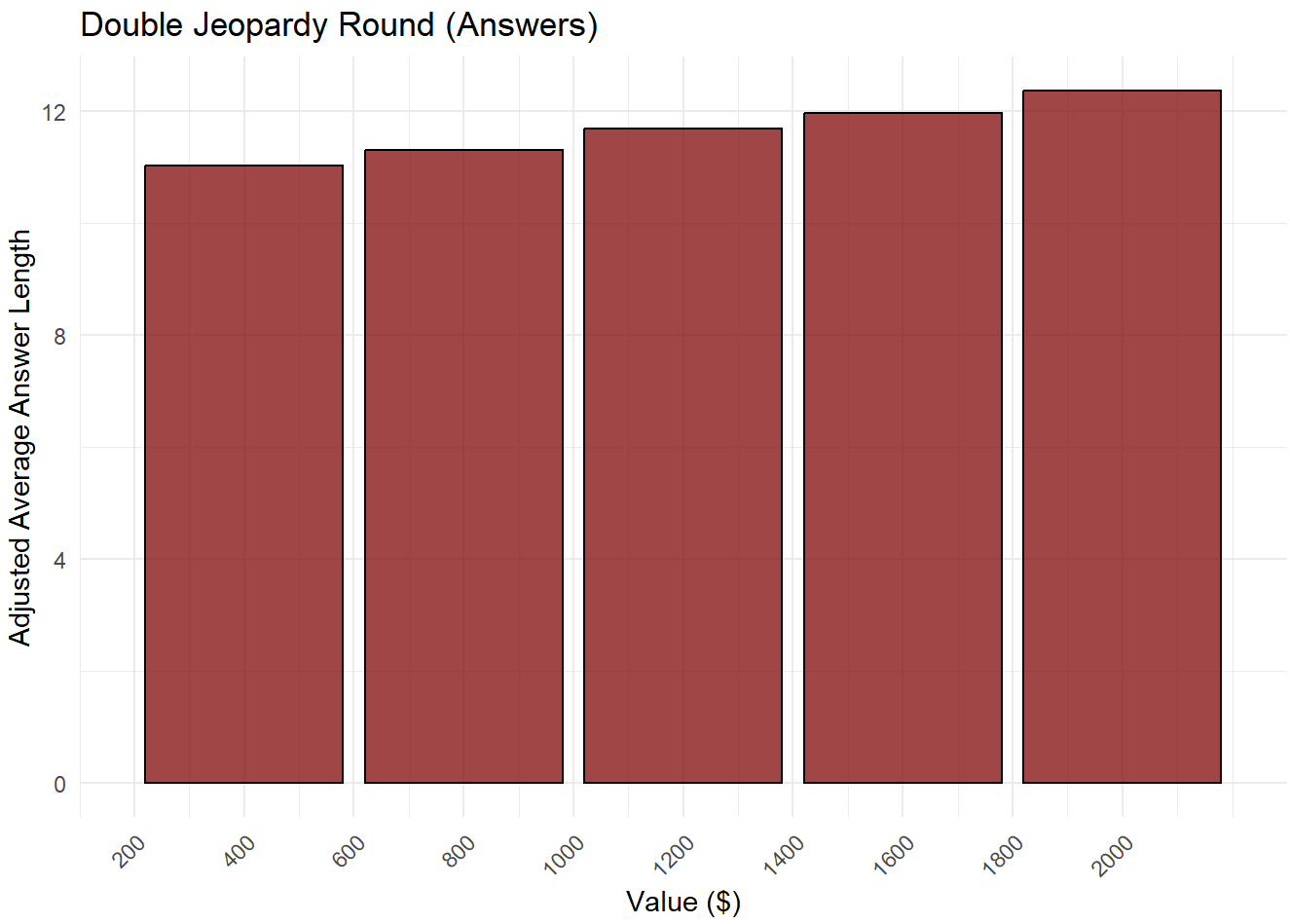
We split the data across the two rounds, Jeopardy and Double Jeopardy, to see if that would have any effect on the results. The resulting charts revealed that there is a positive, linear relationship between the average question length and the value of the question, which was very cool to discover. After that, we decided to see if the trend would persist among the answers of the questions as well.

```
#Rerunning Rohit's average answer length for each value with the adjusted values

adjusted_average_answer_length <- jeopardy_adjusted_data %>%
  mutate(
    AnswerLength = nchar(as.character(Answer)),
    Value = as.numeric(gsub("[\\$,]", "", Value))
  ) %>%
  drop_na(Value) %>%
  filter(Value != 0) %>%
  group_by(Round, Value) %>%
  summarize(
    AverageAnswerLength = mean(AnswerLength, na.rm = TRUE),
    Occurrences = n(),
    .groups = "drop"
  ) %>%
  filter(Occurrences >= 1000)


adjusted_jeopardy_avg <- adjusted_average_answer_length %>%
```

```
  filter(Round == "Jeopardy!")

adjusted_double_jeopardy_avg <- adjusted_average_answer_length %>%
  filter(Round == "Double Jeopardy!")

# Plot for Jeopardy round
plot_adjusted_jeopardy <- ggplot(adjusted_jeopardy_avg, aes(x = Value, y = AverageAnswerLength)) +
  geom_col(fill = "firebrick2", color = "black", alpha = 0.8) +
  labs(
    title = "Jeopardy Round (Answers)",
    x = "Value ($)",
    y = "Adjusted Average Answer Length"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = seq(0, 1000, by = 100)) + # Set x-axis breaks up to 1000
  coord_cartesian(xlim = c(100, 1100)) +                # Cap x-axis at 1000
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

# Plot for Double Jeopardy round
plot_adjusted_double_jeopardy <- ggplot(adjusted_double_jeopardy_avg, aes(x = Value, y = AverageAn
  geom_col(fill = "firebrick4", color = "black", alpha = 0.8) +
  labs(
    title = "Double Jeopardy Round (Answers)",
    x = "Value ($)",
    y = "Adjusted Average Answer Length"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = seq(0, 2000, by = 200)) + # Set x-axis breaks up to 2000
  coord_cartesian(xlim = c(200, 2200)) +                # Cap x-axis at 2000
  theme(axis.text.x = element_text(angle = 45, hjust = 1))


plot_adjusted_jeopardy
```




```
plot_adjusted_double_jeopardy
```



Surely enough, once again the charts showed us that there was a positive, linear correlation between the Average Answer Length and the Value of the question.

Tokenizing

The next step in our process was to tokenize all of the words in the questions and answers and remove any stop words from the data set. A stop word is a word that commonly occurs in a body of text, but has very little information or value to offer. The most common stop words are "The", "a", "is", "and", and so forth. Since the questions and answers in our dataset are written in plain English, there are lots of stopwords that we needed to remove to improve the quality of our data for natural language processing. We used the "stop_words" data set from the tidytext package to remove any stopwords from the dataset.

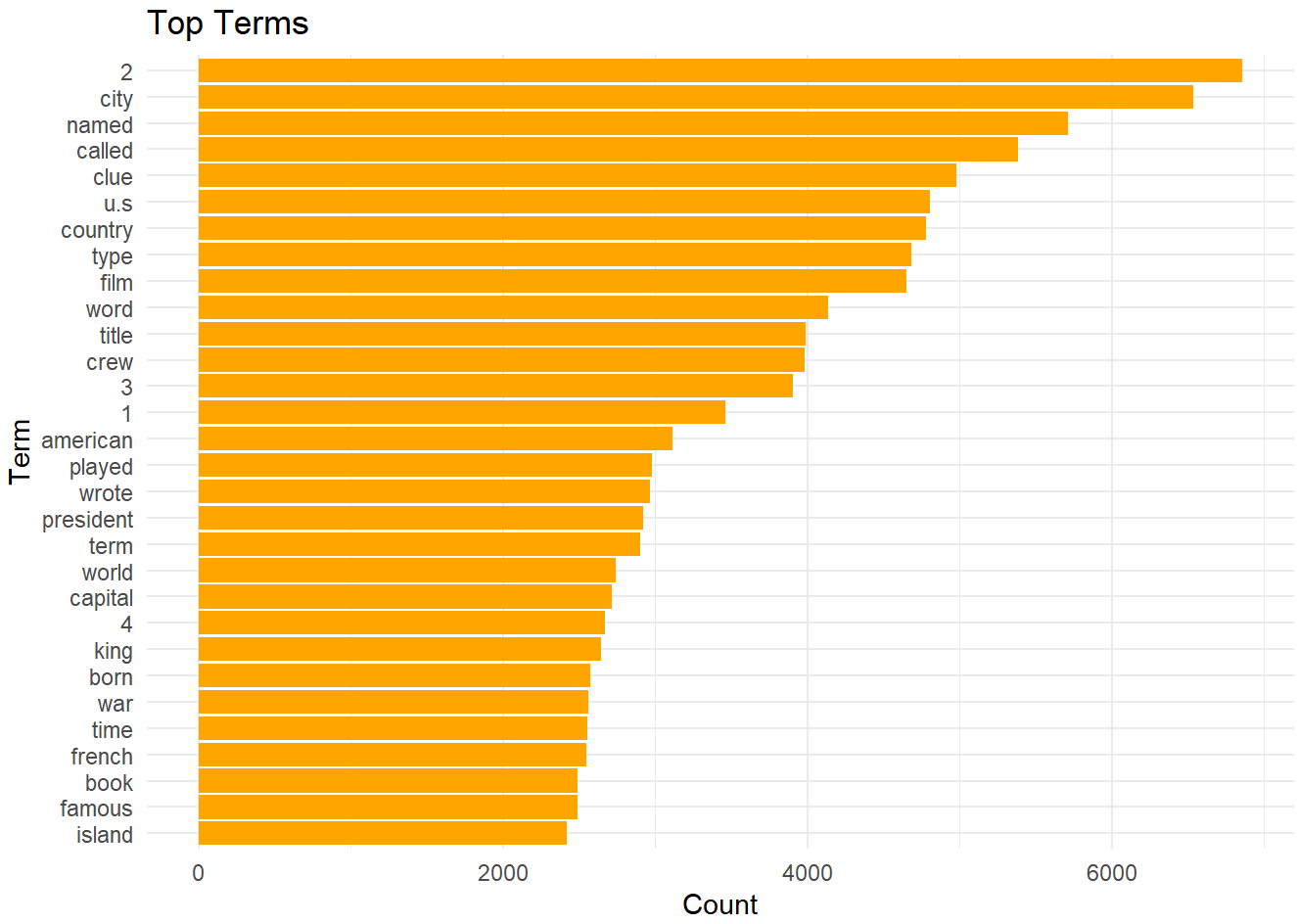
After tokenizing our questions and answers, we were able to create a bar chart of the most commonly occurring words in the dataset.

```
library(tidytext)

data("stop_words")

word_counts <- jeopardy_adjusted_data %>%
  unnest_tokens(word, Question) %>%
  anti_join(stop_words, by = "word") %>%
```

```
  count(word, sort = TRUE)

# Display the top 30 most frequent words
top_words <- head(word_counts, 30)
print(top_words)
```

```
        word    n
1          2 6854
2       city 6538
3      named 5712
4     called 5386
5       clue 4979
6        u.s 4802
7    country 4778
8       type 4686
9       film 4652
10      word 4135
11     title 3990
12      crew 3982
13         3 3903
14         1 3464
15  american 3113
16    played 2976
17     wrote 2968
18 president 2920
19      term 2904
20     world 2742
21   capital 2717
22         4 2669
23      king 2641
24      born 2575
25       war 2562
26      time 2557
27    french 2547
28      book 2490
29    famous 2487
30    island 2420
```

```
# Count bar plot
ggplot(top_words, aes(x = reorder(word, n), y = n)) +
  geom_bar(stat = "identity", fill = "orange") +
  coord_flip() +
  labs(title = "Top Terms", x = "Term", y = "Count") +
  theme_minimal()
```



Modeling

We used Python for modeling, so the code and ouput will be attached here.

Methods

We went through several modeling ideas varying in complexity. Eventually we settled on Logistic Regression. Even though Value was a number, we needed to treat it as categorical to ensure the predictions were correct. It would not have made sense if the category was predicted to be an in-between number like 250. First we combined the Question and Answer columns into one column, called 'cleaned_combined_text'. We used this column as our textual data. We then split the dataset into a Jeopardy dataset and a Double Jeopardy dataset. We initially tried keeping both as part of the same set and using Round as a variable, but realized that this would make for more accurate predictions. We used TF-IDF Vectorization and split both datasets with an 80/20 ratio. The Value column was encoded to a smaller integer. We then trained both models using Logistic Regression based on the TF-IDF vectors. Then we used confusion matrices to better understand the results.

Below is the Logistic Regression Workflow in Python.

```
jeopardy_data = jeopardy_adjusted_data[jeopardy_adjusted_data['Round'] == 'Jeopardy!'].copy()
double_jeopardy_data = jeopardy_adjusted_data[jeopardy_adjusted_data['Round'] == 'Double
Jeopardy!'].copy()
```

```
jeopardy_data = jeopardy_data[jeopardy_data['Value'].isin(valid_values)]
double_jeopardy_data = double_jeopardy_data[double_jeopardy_data['Value'].isin(valid_values)]

# Verify the sizes of the datasets
print(f"Jeopardy dataset size: {jeopardy_data.shape}")
print(f"Double Jeopardy dataset size: {double_jeopardy_data.shape}")
```

Jeopardy dataset size: (106425, 9) Double Jeopardy dataset size: (101735, 9)

```
# For the jeopardy round
jeopardy_data = jeopardy_data[~jeopardy_data['Value'].isin([2000, 1600, 1200])]

# For the double jeopardy round
double_jeopardy_data = double_jeopardy_data[~double_jeopardy_data['Value'].isin([1000, 600,
200])]

# To confirm the filtering, you can print value counts
print("Filtered Jeopardy 'Value' counts:")
print(jeopardy_data['Value'].value_counts())

print("\nFiltered Double Jeopardy 'Value' counts:")
print(double_jeopardy_data['Value'].value_counts())
```

Filtered Jeopardy ‘Value’ counts: Value 200.0 21683 400.0 21467 1000.0 21326 600.0 20794 800.0 20267 Name: count, dtype: int64

Filtered Double Jeopardy ‘Value’ counts: Value 400.0 21466 800.0 20834 2000.0 20731 1200.0 19503 1600.0 18600 Name: count, dtype: int64

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import LabelEncoder

# TF-IDF Vectorization for Jeopardy
tfidf_vectorizer_jeop = TfidfVectorizer(max_features=5000, stop_words='english')
X_jeopardy = tfidf_vectorizer_jeop.fit_transform(jeopardy_data['cleaned_combined_text'])

# TF-IDF Vectorization for Double Jeopardy
tfidf_vectorizer_dj = TfidfVectorizer(max_features=5000, stop_words='english')
X_double_jeopardy =
tfidf_vectorizer_dj.fit_transform(double_jeopardy_data['cleaned_combined_text'])


# Encode 'Value' for Jeopardy
label_encoder_jeop = LabelEncoder()
y_jeopardy = label_encoder_jeop.fit_transform(jeopardy_data['Value'])
```

```
# Encode 'Value' for Double Jeopardy
label_encoder_dj = LabelEncoder()
y_double_jeopardy = label_encoder_dj.fit_transform(double_jeopardy_data['Value'])

# Split Jeopardy data
X_train_jeop, X_test_jeop, y_train_jeop, y_test_jeop = train_test_split(X_jeopardy, y_jeopardy,
test_size=0.2, random_state=42)

# Split Double Jeopardy data
X_train_dj, X_test_dj, y_train_dj, y_test_dj = train_test_split(X_double_jeopardy,
y_double_jeopardy, test_size=0.2, random_state=42)

# Train logistic regression on Jeopardy data
lr_jeop = LogisticRegression(max_iter=1000, random_state=42)
lr_jeop.fit(X_train_jeop, y_train_jeop)

# Train logistic regression on Double Jeopardy data
lr_dj = LogisticRegression(max_iter=1000, random_state=42)
lr_dj.fit(X_train_dj, y_train_dj)

# Evaluate Jeopardy model
y_pred_jeop = lr_jeop.predict(X_test_jeop)
print("Jeopardy Classification Report:")
print(classification_report(y_test_jeop, y_pred_jeop,
target_names=label_encoder_jeop.classes_.astype(str)))
print("Jeopardy Accuracy:", accuracy_score(y_test_jeop, y_pred_jeop))

# Evaluate Double Jeopardy model
y_pred_dj = lr_dj.predict(X_test_dj)
print("\nDouble Jeopardy Classification Report:")
print(classification_report(y_test_dj, y_pred_dj,
target_names=label_encoder_dj.classes_.astype(str)))
print("Double Jeopardy Accuracy:", accuracy_score(y_test_dj, y_pred_dj))
```

Jeopardy Classification Report: precision recall f1-score support

```
   200.0       0.25      0.29      0.27      4305
   400.0       0.21      0.20      0.20      4360
   600.0       0.20      0.18      0.19      4138
   800.0       0.19      0.17      0.18      4017
  1000.0       0.24      0.27      0.26      4288

accuracy                           0.22     21108
```

macro avg 0.22 0.22 0.22 21108 weighted avg 0.22 0.22 0.22 21108

Jeopardy Accuracy: 0.22152738298275534

Double Jeopardy Classification Report: precision recall f1-score support

```
   400.0       0.27      0.33      0.30      4361
   800.0       0.21      0.21      0.21      4131
  1200.0       0.19      0.16      0.17      3884
  1600.0       0.19      0.15      0.17      3748
  2000.0       0.26      0.30      0.28      4103

accuracy                           0.23     20227
```

macro avg 0.23 0.23 0.23 20227 weighted avg 0.23 0.23 0.23 20227

Double Jeopardy Accuracy: 0.23389528847579968

```
# Predict values for Jeopardy
jeopardy_pred_labels = lr_jeop.predict(X_test_jeop)

# Predict values for Double Jeopardy
double_jeopardy_pred_labels = lr_dj.predict(X_test_dj)

# Map predictions and true values back to original categories for Jeopardy
jeopardy_true_values = label_encoder_jeop.inverse_transform(y_test_jeop)
jeopardy_predicted_values = label_encoder_jeop.inverse_transform(jeopardy_pred_labels)

# Map predictions and true values back to original categories for Double Jeopardy
double_jeopardy_true_values = label_encoder_dj.inverse_transform(y_test_dj)
double_jeopardy_predicted_values =
label_encoder_dj.inverse_transform(double_jeopardy_pred_labels)

import pandas as pd

# Jeopardy predictions DataFrame
jeopardy_results = pd.DataFrame({
    'True Value': jeopardy_true_values,
    'Predicted Value': jeopardy_predicted_values
})

# Double Jeopardy predictions DataFrame
double_jeopardy_results = pd.DataFrame({
    'True Value': double_jeopardy_true_values,
    'Predicted Value': double_jeopardy_predicted_values
})

# Preview Jeopardy predictions
print("Jeopardy Predictions:")
print(jeopardy_results.head())

# Preview Double Jeopardy predictions
print("\nDouble Jeopardy Predictions:")
print(double_jeopardy_results.head())
```

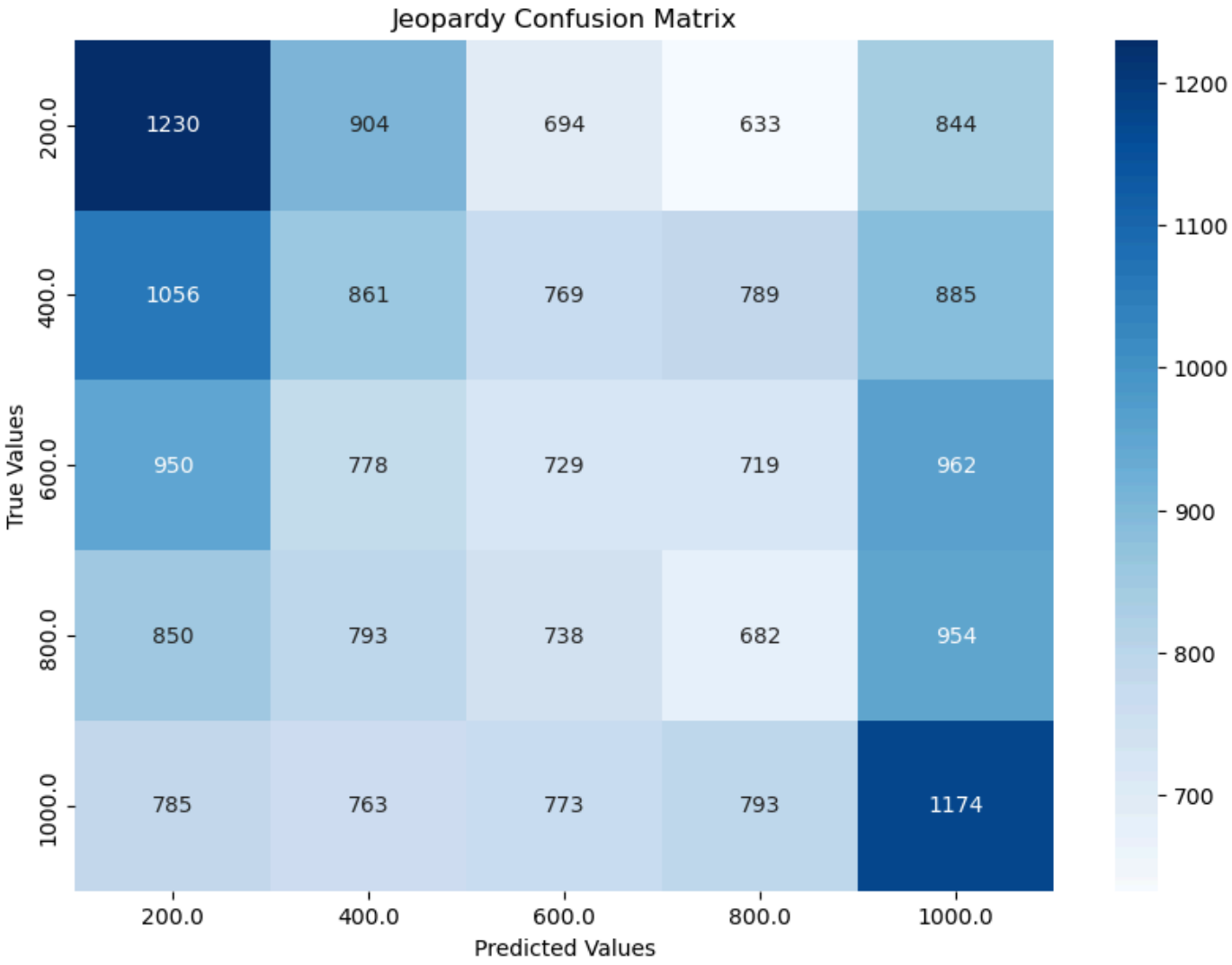
Jeopardy Predictions: True Value Predicted Value 0 600.0 400.0 1 600.0 800.0 2 800.0 400.0 3 1000.0 400.0 4 400.0 800.0

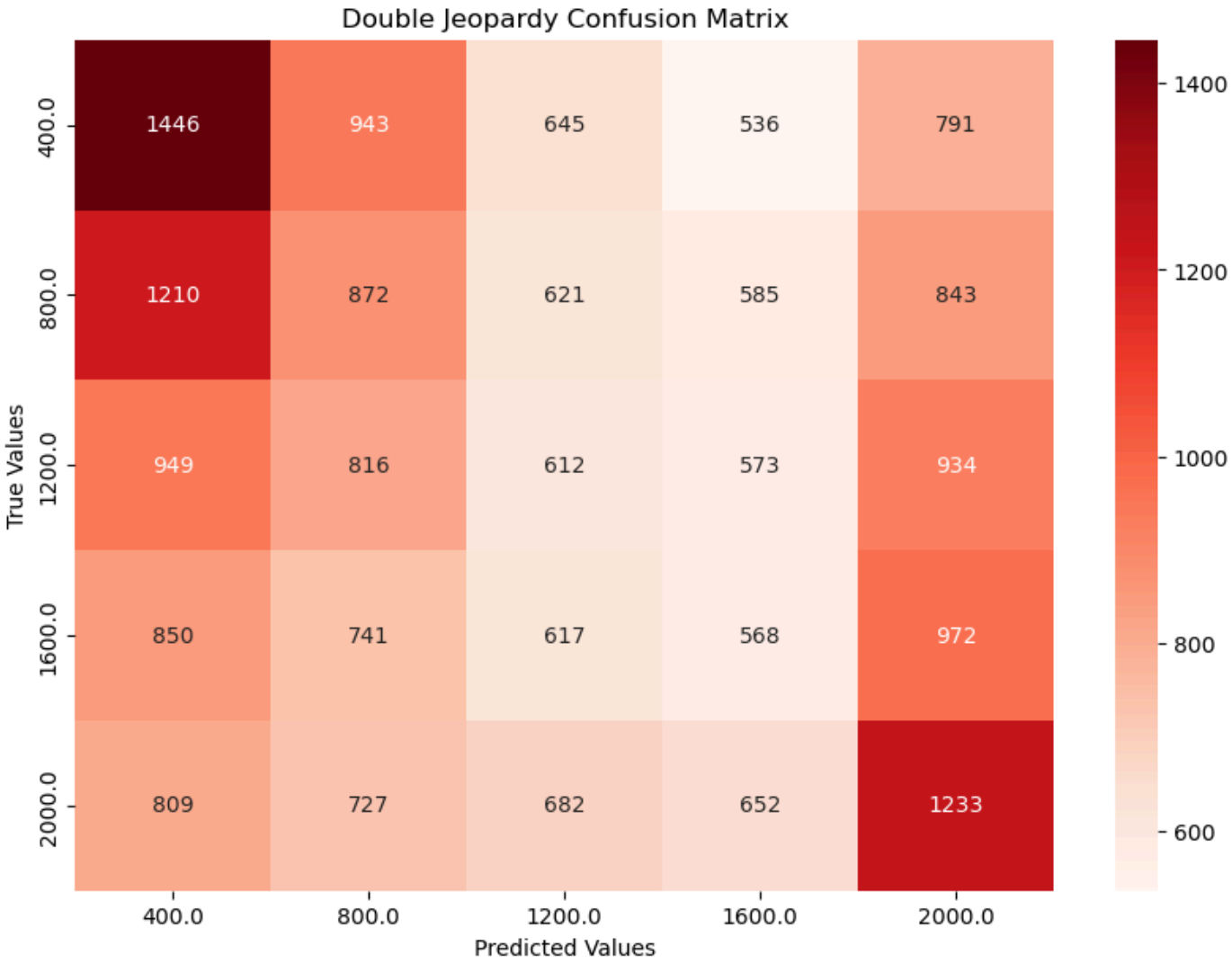
Double Jeopardy Predictions: True Value Predicted Value 0 400.0 1200.0 1 800.0 1200.0 2 400.0 400.0 3 2000.0 800.0 4 400.0 400.0

```
import matplotlib.pyplot as plt
import seaborn as sns

# Generate confusion matrix for Jeopardy
jeopardy_cm = confusion_matrix(jeopardy_true_values, jeopardy_predicted_values)

# Generate confusion matrix for Double Jeopardy
double_jeopardy_cm = confusion_matrix(double_jeopardy_true_values,
double_jeopardy_predicted_values)

# Function to plot heatmap
def plot_confusion_matrix(cm, title, labels, color):
    plt.figure(figsize=(10, 7))
    sns.heatmap(cm, annot=True, fmt='d', cmap=color, xticklabels=labels, yticklabels=labels)
    plt.title(title)
    plt.xlabel('Predicted Values')
    plt.ylabel('True Values')
    plt.show()

# Get the category labels for Jeopardy and Double Jeopardy
jeopardy_labels = label_encoder_jeop.classes_
double_jeopardy_labels = label_encoder_dj.classes_

# Plot Jeopardy confusion matrix
plot_confusion_matrix(jeopardy_cm, "Jeopardy Confusion Matrix", jeopardy_labels, 'Blues')

# Plot Double Jeopardy confusion matrix
plot_confusion_matrix(double_jeopardy_cm, "Double Jeopardy Confusion Matrix",
double_jeopardy_labels, 'Reds')
```

Results

The results were not what we hoped for. No matter how many methods we tried, we could never get an accuracy greater than 0.25 for both datasets. These numbers would be because the model had a tendency to predict either the lowest or highest value for each combination. Using Logistic Regression resulted in an accuracy of 0.225 for the Jeopardy dataset and 0.235 for the Double Jeopardy dataset. While these are lower than our best predictions from other methods, the predictions were more balanced and it felt like the model was not forced to pick the lowest or the highest value. However, it did still have the tendency to pick either 200 or 1000 for the Jeopardy set, and 400 and 2000 for the Double Jeopardy Set.


Jeopardy Confusion Matrix
True Values
Predicted Values
200.0
400.0
600.0
800.0
1000.0
1230
904
694
633
844
1056
861
769
789
885
950
778
729
719
962
850
793
738
682
954
785
763
773
793
1174
1200
1100
1000
900
800
700



Conclusions

In conclusion, it is difficult to say that there is any real connection between the text in the Questions and Answers in Jeopardy, and their Value. There were some predictions that were correct, but it felt almost as if the predictions were random guessing. There is still a clear correlation between the length of the text and the Value, so for future attempts, we could incorporate that into the model. There are also several other, higher-complexity, models we could use that could potentially make better predictions.